

Generators & Iterators

Memory-efficient data streaming with yield

■ What Are Generators and Iterators?

Generators and Iterators are one of the most powerful features for memory management and performance in Python backend development.

When you create a standard list in Python, memory is allocated for every single item at once:

```
# Memory Heavy: creates a list of 1,000,000 numbers in memory immediately
numbers = [x for x in range(1000000)]
```

A **Generator** does not store all items in memory. Instead, it **yields** items one at a time, on demand, as you iterate over it.

The yield Keyword vs return

- **return** — terminates the function and sends back a final value.
- **yield** — pauses the function, saves its state, returns a value, and waits for the next call!

■■ Comparing Memory Usage: List vs Generator

A Generator Expression uses `()` instead of `[]`:

```
import sys

# List Comprehension
list_data = [x for x in range(10000)]
print("List Size:", sys.getsizeof(list_data), "bytes")

# Generator Expression
gen_data = (x for x in range(10000))
print("Generator Size:", sys.getsizeof(gen_data), "bytes")
```

■ Output Preview

The list takes tens of thousands of bytes, while the generator takes only ~200 bytes — regardless of whether it's counting to 10 or 10,000,000!

Day 75 — Generators & Iterators

■ The Popcorn Machine Analogy

Generators can feel abstract at first, so here's an everyday story: imagine you want 100 bags of popcorn for a party.

1. The Regular List Way — The Storage Room

- You pop all 100 bags at once.
- You need a huge table (RAM / Memory) to hold all 100 bags before anyone even eats one.
- If you ask for 1,000,000 bags, your room runs out of space and explodes (Out of Memory Error)!

2. The Generator Way — The Fresh Popcorn Machine

- The machine doesn't store anything.
- You step up and ask: "Give me popcorn!"
- The machine pops 1 bag, hands it to you (yield), and pauses.
- It waits until you ask for the next bag before doing any work.
- Space needed? Just enough for 1 single bag, no matter if you want 10 or 10,000,000 bags!

■ The Single Magic Word: yield

In regular Python functions, you use return:

```
def make_list():  
    return [1, 2, 3] # Done! Hands over everything and shuts down the function.
```

In a Generator, you use yield:

```
def make_generator():  
    yield 1 # Pauses here, hands over 1  
    yield 2 # Wakes up, hands over 2, pauses again  
    yield 3 # Wakes up, hands over 3
```

When Python sees `yield`, it doesn't kill the function. It just pauses it until you ask for the next item!

■ Simplified First Script

Day 75 — Generators & Iterators

A super short generator, run one step at a time with `next()`:

```
# A super simple generator function
def popcorn_machine():
    print("■ Popping Bag 1...")
    yield "Bag 1"

    print("■ Popping Bag 2...")
    yield "Bag 2"

# Start the machine
snack = popcorn_machine()

# Ask for bags one by one using next()
print(next(snack))
print(next(snack))
```

■ What `next()` does behind the scenes

First `next(snack)`: Python enters `popcorn_machine()`, prints "Popping Bag 1...", hits `yield "Bag 1"`, hands you "Bag 1", and freezes the function right on that line.

Second `next(snack)`: Python wakes the function back up right where it paused, moves to the next line, prints "Popping Bag 2...", hits `yield "Bag 2"`, and hands you "Bag 2".

■ Using Generators in for Loops

In real projects, you don't call `next()` manually — you use a standard for loop. The for loop automatically calls `next()` behind the scenes until the generator runs out of items.

The `sys` Module — What It's For

`sys` is a built-in module that lets a script interact directly with the Python interpreter and operating system — a control panel for environment settings, system resources, and inputs/outputs. No `pip install` needed.

- **`sys.getsizeof()`** — bytes of memory an object occupies (used today).
- **`sys.argv`** — reads command-line arguments passed when launching a script.
- **`sys.exit()`** — stops a script immediately.
- **`sys.version` / `sys.platform`** — Python version and OS platform.

Day 75 — Generators & Iterators

Updated day75.py — Loop, Generator Expression, Memory Check

```
# =====
# DAY 75: GENERATORS AND ITERATORS
# =====
import sys

# 1. Custom Generator Function
def popcorn_machine():
    yield "■ Bag 1"
    yield "■ Bag 2"
    yield "■ Bag 3"

print("--- 1. Loop through popcorn_machine() ---")
for bag in popcorn_machine():
    print(bag)

# 2. Generator Expression vs List Comprehension
# Note: Generator uses (), List uses []
list_nums = [x for x in range(1, 6)]
gen_nums = (x for x in range(1, 6))

print("\n--- 2. Generator Expression Output ---")
for num in gen_nums:
    print("Number:", num)

# 3. Why Generators Matter: Memory Check!
big_list = [x for x in range(100000)]
big_gen = (x for x in range(100000))

print("\n--- 3. Memory Usage ---")
print("100,000 items in a List      :", sys.getsizeof(big_list), "bytes")
print("100,000 items in a Generator:", sys.getsizeof(big_gen), "bytes")
```

■ Day 75 Complete: Outstanding Work! ■

List Size: 800,984 bytes (~800 KB) vs Generator Size: 200 bytes (0.2 KB) — the generator handled the exact same 100,000 items using 4,000 times less memory. This is why backend engineers love generators: reading millions of database records or massive log files stays fast without crashing the server.

Day 75 — Generators & Iterators

■ Save & Push Day 75 to GitHub

```
git add .
git commit -m "Day 75: Mastered generators, iterators,
               yield keyword, and memory optimization with sys"
git push
```

■■ Challenge #1: The Server Log Scanner

Write a generator function `error_scanner(logs)` that loops through a list of log strings, checks if "ERROR" is in each string, and yields the ones that are errors.

Test Data

```
server_logs = [
    "[INFO] User logged in",
    "[ERROR] Database connection failed",
    "[INFO] Page requested /home",
    "[ERROR] Out of memory error",
    "[WARN] High CPU usage",
    "[ERROR] 404 Not Found"
]
```

■ Challenge #1 (Revised): Even Number Streamer

Write a generator function `even_streamer(numbers)` that takes a list of numbers, yields only the even ones (`num % 2 == 0`), and is looped through with a for loop that prints each even number.

Test input:

```
test_numbers = [12, 7, 19, 24, 30, 41, 58, 99]
```

Submitted Solution

Day 75 — Generators & Iterators

```
def even_streamer(numbers):
    for num in numbers:
        if num % 2 == 0:
            yield num

text_numbers = [12, 7, 19, 24, 30, 41, 58, 99]

for even_num in even_streamer(text_numbers):
    print(even_num)
```

Terminal Output

```
12
24
30
58
```

■ 100% Perfect! Code Review Passed! ■

Looping, condition, and yield were all spot on — a true generator that filters even numbers on demand.

■ One Micro-Fix Noted

In the memory check line, bytes was written without quotes, so Python printed <class 'bytes'> instead of the word "bytes". Fix: wrap it in quotes — `print(..., sys.getsizeof(big_list), "bytes")`.

■ Challenge #2: The Password Length Filter

Write a generator function `valid_password_streamer(passwords)` that loops through a list of passwords, checks each length with `len()`, and yields only passwords that are 8 characters or longer — shorter ones are simply skipped.

Test Input

Day 75 — Generators & Iterators

```
user_passwords = ["pass123", "SecurePass2026!", "12345", "SuperSecretCode", "admin", "PythonDev789!"]
```

Submitted Solution

```
def valid_password_streamer(passwords):
    for password in passwords:
        if len(password) > 8:
            yield password
        else:
            False

user_passwords = ["pass123", "SecurePass2026!", "12345", "SuperSecretCode", "admin", "PythonDev789!"]

for u_password in valid_password_streamer(user_passwords):
    print(u_password)
```

Terminal Output

```
SecurePass2026!
SuperSecretCode
PythonDev789!
```

■ 2 for 2! Another Perfect Solution! ■

Weak passwords (pass123, 12345, admin) were correctly filtered out, and all three strong passwords streamed through.

■ One Quick Code Tip

The `else: False` branch isn't needed — in a generator, if the condition isn't met, Python just moves on to the next loop iteration automatically. It can be streamlined to just the `if-yield`, no else at all.

■ Summary of What You Mastered Today

- **yield vs return:** how yield pauses execution and saves memory instead of terminating the function.
- **next() & for loops:** pulling data from generators manually and automatically.

Day 75 — Generators & Iterators

- **Memory optimization (`sys.getsizeof`):** generators use tiny amounts of RAM (200 bytes) compared to giant lists.
 - **Custom filters:** built two real-world generator functions (`even_streamer` and `valid_password_streamer`) completely from scratch!
-

■ Final Push to GitHub

```
git add .
git commit -m "Day 75: Passed password streaming generator
               challenge and mastered memory-efficient generators"
git push
```